

Advanced Machine Learning

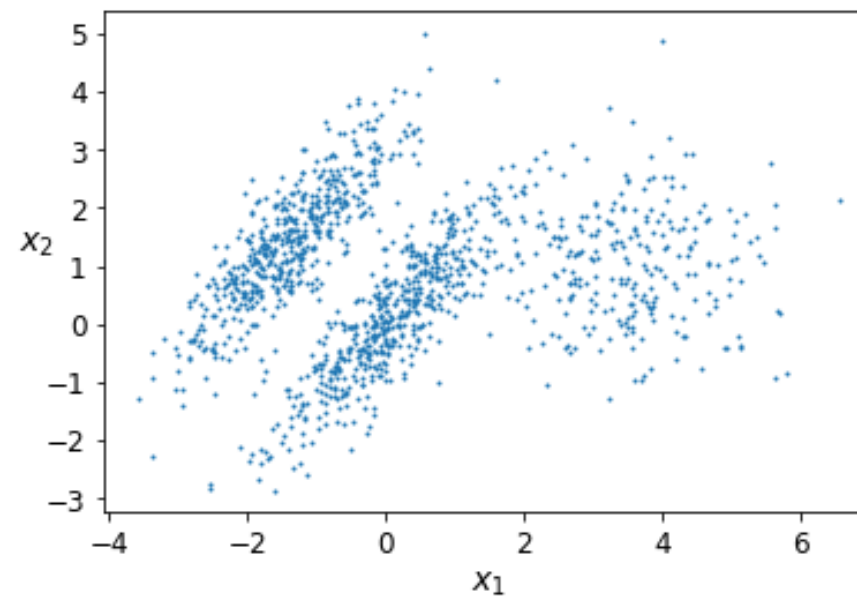
DATA 642

Lecture 11 — Gaussian Mixtures

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Gaussian Mixture Models (GMMs)

- A GMM is a probabilistic model assuming instances were generated from a mixture of Gaussian distributions with unknown parameters.
- All the instances generated from a single Gaussian distribution form a cluster that typically looks like an ellipsoid. Each cluster can have a different ellipsoidal shape, size, density and orientation.



- When you observe an instance, you know it was generated from one of the Gaussian distributions, but you are not told which one, and you do not know what the parameters of these distributions are.

The math behind GMMs

Probability of Cluster Assignment

For each instance $\mathbf{x}_i$, a cluster z_i is randomly picked among k clusters. The probability of choosing the j th cluster is defined by its weight ϕ_j .

$$P(z_i = j) = \phi_j \tag{1}$$

Sampling from Gaussian Distribution

If $z_i = j$, meaning the i th instance is assigned to the j th cluster, the location $\mathbf{x}_i$ of this instance is sampled randomly from a Gaussian distribution with mean $\boldsymbol{\mu}_j$ and covariance matrix $\boldsymbol{\Sigma}_j$.

$$\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \tag{2}$$

This probabilistic process ensures that the dataset $\mathbf{X}$ is generated from a mixture of k Gaussian distributions, where each distribution is associated with a weight ϕ_j , mean $\boldsymbol{\mu}_j$, and covariance matrix $\boldsymbol{\Sigma}_j$.

The goal of GMMs is to estimate these parameters from the observed dataset $\mathbf{X}$.

Log-Likelihood Function in GMMs

- **Log-Likelihood Function**

- The log-likelihood function represents the likelihood of observing the given data under the assumed Gaussian Mixture Model (GMM).
- It is defined as the logarithm of the joint probability of observing both the data $\mathbf{X}$ and the set of latent variables $\mathbf{Z}$, given the model parameters $\boldsymbol{\theta}$.

$$\log p(\mathbf{X}, \mathbf{Z} | \boldsymbol{\theta}) = \sum_{i=1}^n \log p(\mathbf{x}_i, z_i | \boldsymbol{\theta}) = \sum_{i=1}^n \log \left(\sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right) \quad (3)$$

- **Optimization during Parameter Estimation**

- The goal of parameter estimation in GMMs is to maximize the log-likelihood function with respect to the model parameters $\boldsymbol{\theta}$.
- The Expectation-Maximization (EM) algorithm achieves this by iteratively updating the parameters to increase the log-likelihood.
 - At each iteration, the EM algorithm alternates between an Expectation (E) step and a Maximization (M) step, optimizing the log-likelihood until convergence.

Expectation-Maximization (EM) Algorithm

Expectation Step (E-step)

In the E-step, we take a guess about which cluster each data point belongs to. We calculate the probability of each data point belonging to each cluster based on our current model parameters.

$$\gamma^{(t)}(z_{ij}) = \frac{\phi_j^{(t)} \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}{\sum_{j=1}^k \phi_j^{(t)} \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})} \quad (4)$$

Maximization Step (M-step) In the M-step, we update our model's parameters (cluster means, covariances, and weights) to better fit the data. We do this by maximizing the likelihood of observing the data given our current guesses.

Update cluster means:

$$\boldsymbol{\mu}_j^{(t+1)} = \frac{\sum_{i=1}^n \gamma^{(t)}(z_{ij}) \boldsymbol{x}^{(i)}}{\sum_{i=1}^n \gamma^{(t)}(z_{ij})} \quad (5)$$

Update cluster covariances:

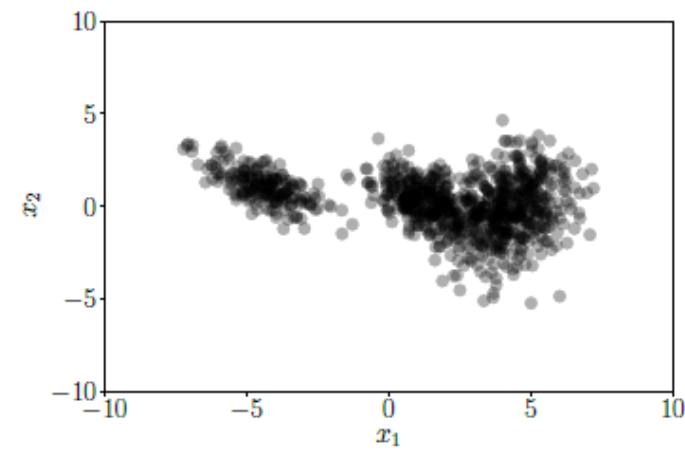
$$\boldsymbol{\Sigma}_j^{(t+1)} = \frac{\sum_{i=1}^n \gamma^{(t)}(z_{ij}) (\boldsymbol{x}^{(i)} - \boldsymbol{\mu}_j^{(t+1)}) (\boldsymbol{x}^{(i)} - \boldsymbol{\mu}_j^{(t+1)})^T}{\sum_{i=1}^n \gamma^{(t)}(z_{ij})} \quad (6)$$

Update cluster weights:

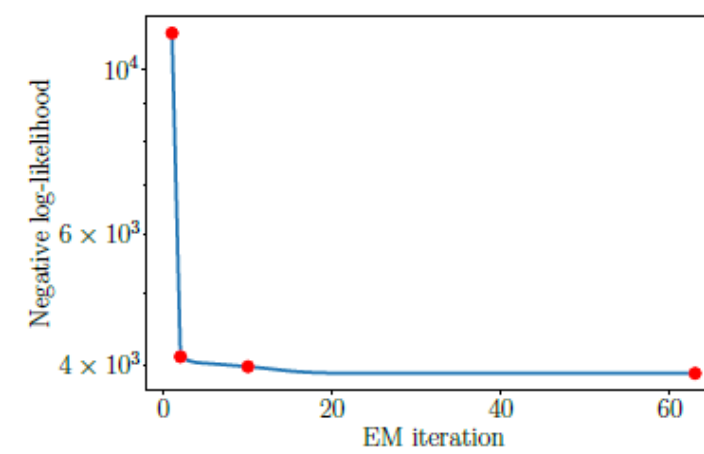
$$\phi_j^{(t+1)} = \frac{\sum_{i=1}^n \gamma^{(t)}(z_{ij})}{n} \quad (7)$$

EM is all about repetition After each E-step and M-step, we have a slightly better understanding of our model and data. So, we keep iterating until we can't improve our model any further. After many iterations, we reach a point where our model parameters stop changing significantly. That's when we know we've found a good set of parameters that maximize the likelihood of observing our data under the GMM.

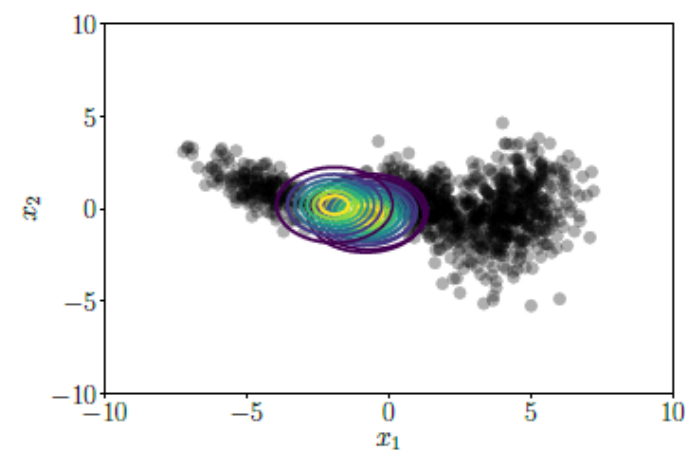
EM in action



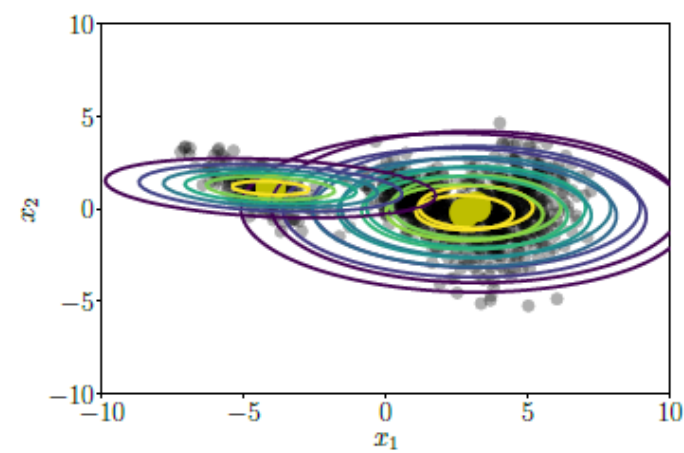
(a) Dataset.



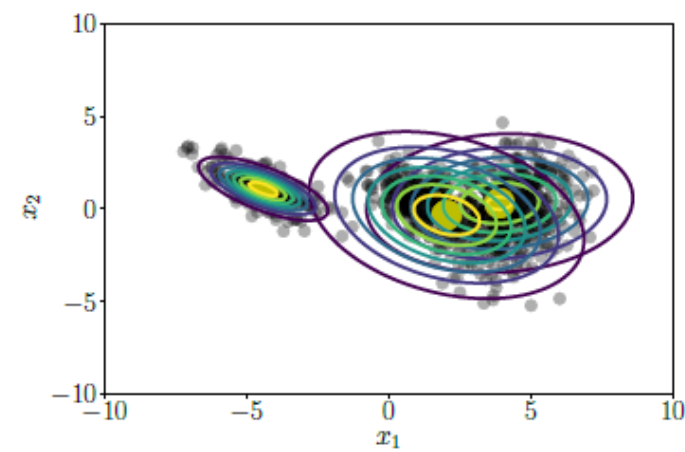
(b) Negative log-likelihood.



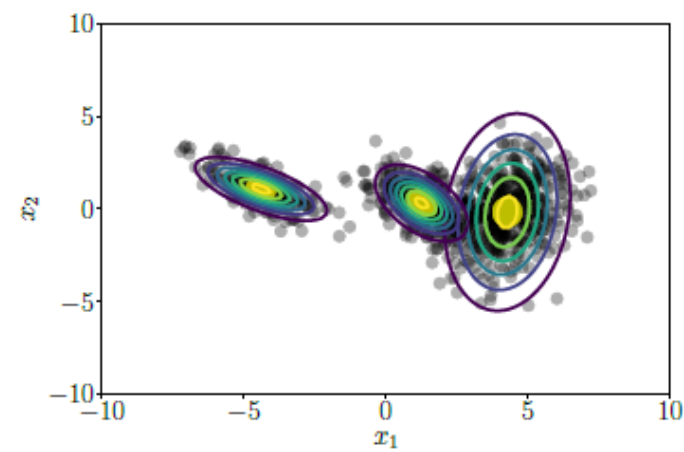
(c) EM initialization.



(d) EM after one iteration.



(e) EM after 10 iterations.



(f) EM after 62 iterations.

Applications and Use Cases

- **Image Segmentation:**

- GMMs are commonly used for image segmentation tasks, where they can identify different regions in an image based on color or texture similarity.
- Advantages:
 - GMMs can capture complex distributions of pixel intensities, making them effective for segmenting images with varying lighting conditions and textures.
 - They provide soft clustering, allowing pixels to belong to multiple segments with different probabilities, which is beneficial for handling ambiguous regions.

- **Anomaly Detection:**

- GMMs are utilized in anomaly detection applications to identify unusual patterns or outliers in data.
- Advantages:
 - GMMs can model the normal behavior of data using multivariate Gaussian distributions and detect deviations from this behavior as anomalies.
 - They offer flexibility in modeling complex data distributions, making them suitable for detecting anomalies in diverse datasets, such as network traffic, financial transactions, or sensor data.

- **Speech Recognition:**

- GMMs play a crucial role in speech recognition systems, where they model the probability distributions of speech features such as Mel-frequency cepstral coefficients (MFCCs).
- Advantages:
 - GMMs can capture the variability in speech sounds and model their statistical properties effectively.
 - They enable the creation of acoustic models that can accurately recognize spoken words and phrases in various environments and accents.

Implementation and Practical Considerations

- **Initialization Methods:**

- Choosing appropriate initialization methods is essential for effective GMM training.
- Common initialization techniques include random initialization, k-means clustering followed by covariance estimation, or hierarchical clustering.
- It's crucial to experiment with different initialization methods to find the one that yields the best performance for a specific dataset.

- **Dealing with Singular Covariance Matrices:**

- GMMs may encounter issues with singular covariance matrices, especially when dealing with high-dimensional data or insufficient training samples per cluster.
- Regularization techniques such as adding a small constant to the diagonal of covariance matrices can help stabilize the estimation process and prevent singularity.
- Alternatively, reducing the dimensionality of the data using techniques like principal component analysis (PCA) may alleviate singularity problems.

- **Model Selection and Evaluation:**

- GMM performance heavily depends on the choice of the number of components (clusters) and model complexity.
- Techniques such as Bayesian information criterion (BIC) or cross-validation can aid in selecting the optimal number of components.
- It's essential to evaluate GMMs using appropriate metrics such as log-likelihood, silhouette score, or clustering purity to assess their effectiveness in capturing underlying data patterns.

- **Computational Efficiency:**

- GMM training can be computationally intensive, especially for large datasets or high-dimensional feature spaces.
- Utilizing efficient implementations provided by libraries like scikit-learn or TensorFlow can help improve training speed.
- Additionally, techniques such as mini-batch learning or parallelization can be employed to accelerate the training process and handle larger datasets more efficiently.